Design.pdf - EV Dataset Schema and Performance Analysis

1. Row Key and Sharding

Proposed Row Key

Row key: salt#city#make#vin10

I propose a composite row key with a short random or hashed salt at the beginning, followed by City, Make, and the VIN (1-10) field. A simple example is *07#Seattle#TESLA#5YJ3E1EA*.

This design reduces hotspotting because new rows are not written to only one small key range. The salt spreads writes across many tablets, so the system can distribute traffic to different nodes. If the row key started only with City or only with Model Year, many new writes could go to the same region and create an uneven load.

The effect on queries is mixed, which is normal in Bigtable. This key is still useful for queries by City and Make because those values are inside the key, but the salt means one request may need to check several salted prefixes. In practice, this is a trade-off: better write distribution, but slightly more complex query routing. If the workload needs very fast lookups by City only or Make only, I would add secondary index tables such as *city#salt#vin* or *make#salt#vin*.

2. Column Families

A good Bigtable schema should group attributes that are read together. This improves distributed read performance because the system can read only the needed families instead of scanning every value in the row.

Column Family	Suggested Fields	Reason
vehicle_info	VIN (1-10), Make, Model, Model Year, Electric Vehicle Type, Electric Range	Core vehicle identity and technical details are often read together.
location_info	County, City, State, Postal Code, Vehicle Location, 2020 Census Tract, Legislative District	Supports geographic analysis, mapping, and local reporting.
policy_registration	CAFV Eligibility, DOL Vehicle ID, Electric Utility	Administrative and policy fields can be read separately from vehicle details.

With this design, an analytical read that needs only city-level information can focus on *location_info*. A dashboard about EV brands can focus on *vehicle_info*. This lowers unnecessary I/O and helps the cluster serve more reads in parallel.

3. Horizontal Scaling: Hashed vs Range-Based Shard Keys

Hashed shard key on VIN

- A hashed shard key on VIN gives a very even distribution of writes and avoids hotspotting. This is good for cluster balance and for high write throughput.
- The main disadvantage is query routing. Range queries are weak because nearby VIN values do not stay near each other after hashing. A query by Model Year or by a time-like range would probably touch many shards.

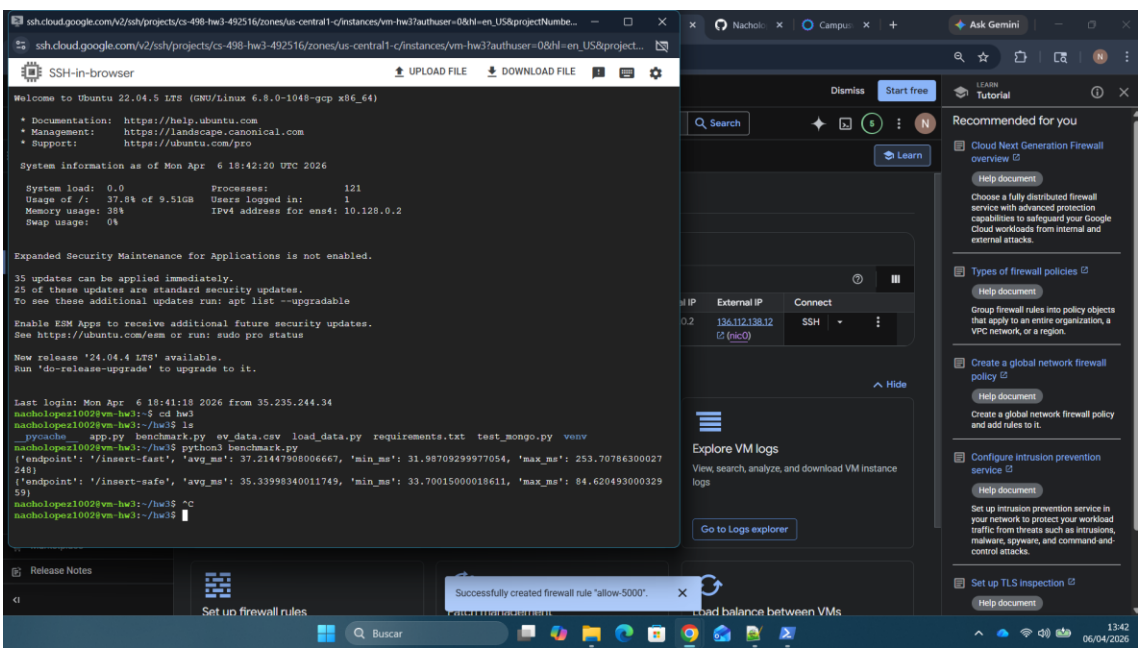
Range-based shard key on Model Year

- A range-based key on Model Year is better for targeted queries such as 2022-2024 vehicles. The router can send the query to fewer shards, so the request can be more efficient.
- The risk is uneven load. If most new data belongs to recent years, the newest range may receive most writes and become a hotspot. This can reduce cluster performance even if query routing is simpler.

Overall, I would choose the hashed design when the system expects heavy write traffic and many parallel inserts. I would choose the range-based design when the workload is mostly analytical and strongly centered on year-based filters.

4. Performance Analysis

Measured latencies from 50 requests per endpoint are shown below.



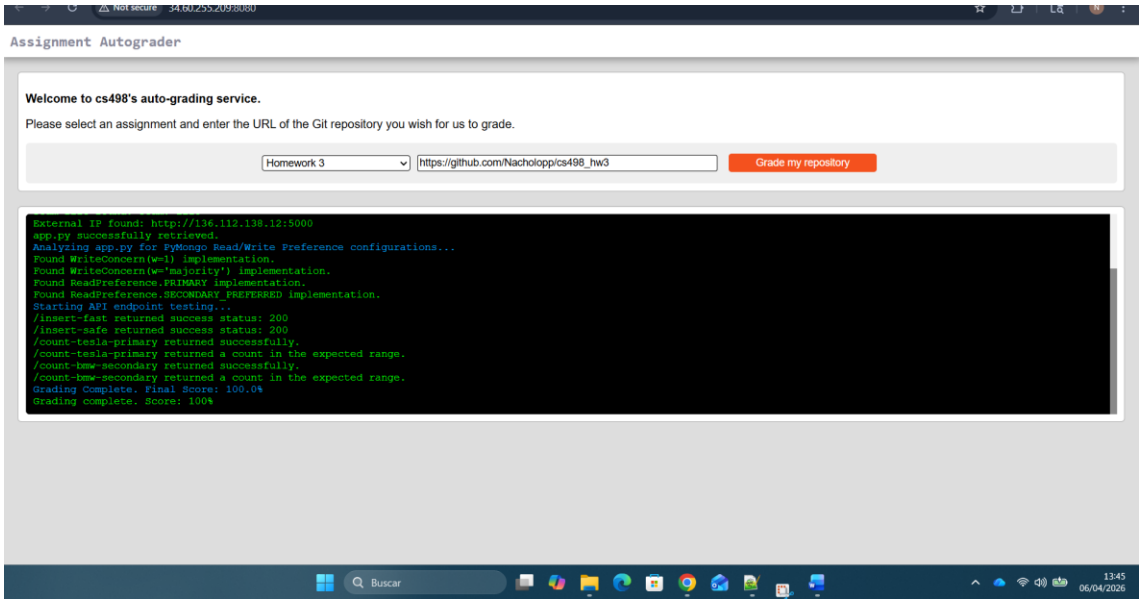
Endpoint	Average latency (ms)	Minimum (ms)	Maximum (ms)
/insert-fast	37.21	31.99	253.71
/insert-safe	35.34	33.70	84.62

The benchmark shows very similar average latency for both endpoints, and in this specific run the majority write was slightly faster. I would not conclude that majority writes are always faster. The difference is small and may come from normal cloud variation, network jitter, temporary caching, or background load on the VM and the Atlas cluster. From a systems point of view, *insert-safe* still has the stricter durability requirement because it

waits for acknowledgement from a majority of replica set members.

CAP theorem scenario

A real-world case where I would still choose the faster primary-only write is clickstream or user activity logging for a shopping website. These events are useful for analytics and recommendation models, but losing a small number of log records is usually acceptable. In that case, low latency and high throughput are more important than perfect durability for every single event. The business keeps the customer experience fast, and the system can accept a small risk of data loss if the primary crashes before replication finishes.

Conclusion

For this EV dataset, a salted composite row key gives a balanced solution: it improves write distribution, keeps the schema flexible, and can still support common query patterns with extra index tables when needed. Column families should separate vehicle, location, and policy data to reduce unnecessary reads. Finally, the benchmark confirms the general trade-off of tunable consistency: latency differences may change from run to run, but stronger durability still provides safer writes in distributed systems.